Dispositivos Conectados
Modos de consumo de energia reduzido

2026/2027

Authors: Paulo C. Bartolomeu

Versão 1.0
3 de novembro de 2026

Histórico de versões

Versão	Data	Alterações
1.0	2026-11-03	Versão inicial.

1 Objetivos

- Familiarização com os modos de operação do ESP32-C6;
- Utilização dos modos de consumo *light-sleep* e *deep-sleep*;
- Utilização dos modos de *wake-up* através do *timer*, de GPIOs e do sinal EXT1;
- Medição da corrente e da carga consumidas em cada fase de funcionamento com o *Power Profiler Kit II* e estimativa da autonomia de um nó alimentado por bateria.

2 Introdução

Em aplicações alimentadas por bateria, como os sensores de monitorização ambiental, a autonomia depende de o microcontrolador passar a maior parte do tempo num modo de consumo reduzido.

O ESP32-C6 tem dois núcleos RISC-V de 32 bits: um núcleo de alto desempenho (HP, até 160 MHz), onde corre a aplicação, e um núcleo de baixo consumo (LP, até 20 MHz), que pode continuar a funcionar enquanto o resto do *chip* dorme. A energia é gerida por domínios: o sistema HP (CPU HP, memória principal, periféricos digitais, rádios Wi-Fi 6, Bluetooth LE e IEEE 802.15.4) pode ser parado ou desligado, enquanto o sistema LP (gestor de energia, *timer* LP, memória LP, LP GPIO e núcleo LP) permanece alimentado.

3 Modos de baixo consumo

A Tabela 2 resume os modos de energia do ESP32-C6 e os consumos típicos indicados na ficha técnica. Estes valores referem-se apenas ao *chip*: um módulo ou uma placa somam o consumo da memória *flash*, do regulador, dos LEDs e de outros componentes.

Tabela 2: Modos de energia do ESP32-C6 e consumos típicos da ficha técnica

Modo	O que fica ativo	Consumo típico do *chip*
Ativo	CPU HP, periféricos e rádio a transmitir ou receber	centenas de mA durante a transmissão Wi-Fi (até cerca de 350 mA à potência máxima)
Modem-sleep	CPU HP e periféricos; o rádio só é ligado quando necessário	até cerca de 38 mA, com a CPU a 160 MHz
Light-sleep	CPU HP parada e periféricos sem relógio; RAM e estado preservados	180 µA; 35 µA com os periféricos digitais desligados
Deep-sleep	Apenas o sistema LP (*timer*, memória LP, LP GPIO e, opcionalmente, o núcleo LP)	7 µA
Power off	Nada; o pino CHIP_PU está a 0 e só um reinício externo acorda o *chip*	cerca de 1 µA

Em **light-sleep** e em **deep-sleep** os rádios ficam desligados e as ligações Wi-Fi ou BLE não se mantêm. Ao acordar do *light-sleep*, o programa continua na instrução seguinte, sem reinício; o ESP-IDF v6.1 pode ainda desligar a CPU e os periféricos digitais durante o sono, guardando e restaurando o seu contexto, se a aplicação o permitir (Questão 5). Ao acordar do *deep-sleep*, o *chip* reinicia e a aplicação volta a começar em `app_main()`, mas pode recuperar dados guardados na memória LP. O núcleo LP, programado em C (componente `ulp`), pode ler sensores durante o *deep-sleep* e acordar o núcleo HP, mas não é usado nesta aula.

As fontes de *wake-up* são geridas pelo sistema LP:

- **Timer**: acorda o *chip* após um intervalo pré-definido (*light-sleep* e *deep-sleep*).
- **EXT1**: acorda o *chip* quando um ou mais **LP GPIO (GPIO0 a GPIO7)** atingem um nível lógico (*light-sleep* e *deep-sleep*). No ESP32-C6 os modos são `ESP_EXT1_WAKEUP_ANY_LOW` (qualquer pino a 0) e `ESP_EXT1_WAKEUP_ANY_HIGH` (qualquer pino a 1), e o nível pode ser escolhido pino a pino. O modo `ESP_EXT1_WAKEUP_ALL_LOW` (todos os pinos a 0) só existe no ESP32 original e está obsoleto no ESP-IDF v6.1. O EXT1 é sensível ao nível, não à transição.
- **GPIO**: em *light-sleep*, qualquer GPIO pode acordar o *chip* (`gpio_wakeup_enable()` e `esp_sleep_enable_gpio_wakeup()`). Em *deep-sleep*, e em *light-sleep* com os periféricos digitais desligados, só os LP GPIO o podem fazer; a função a usar nesse caso é `esp_sleep_enable_gpio_wakeup_on_hp_periph_powerdown()`.
- **UART** (apenas *light-sleep*) e **núcleo LP**.

O ESP32-C6 não tem sensor tátil, pelo que o *wake-up* por toque, disponível noutros *chips* da família, não se aplica. Depois de acordar, a função `esp_sleep_get_wakeup_causes()` devolve um mapa de bits com as causas do *wake-up*, em que cada causa se testa com uma máscara, por exemplo `BIT(ESP_SLEEP_WAKEUP_TIMER)` para o *timer*. A função `esp_sleep_get_wakeup_cause()` está obsoleta no ESP-IDF v6.1.

4 Medição de corrente com o Power Profiler Kit II

O *Nordic Power Profiler Kit II* (PPK2) mede correntes entre cerca de 200 nA e 1 A, a até 100 000 amostras por segundo, e tem um porto lógico com 8 entradas digitais (D0 a D7). Pode funcionar

como fonte de alimentação (*Source meter*) ou como amperímetro (*Ampere meter*). Nesta aula usa-se o modo amperímetro: a placa continua a ser alimentada pela porta USB e a corrente do módulo atravessa o PPK2, de VIN para VOUT. A resolução depende da gama, que o PPK2 escolhe automaticamente: 0,2 µA abaixo de 50 µA, mas 50 µA entre 5 e 50 mA. A exatidão do valor médio é de cerca de ±10%.

A ESP32-C6-DevKitC-1 tem um conector J5 destinado à medição de corrente. Sem o *jumper*, a alimentação de 3,3 V do módulo ESP32-C6-WROOM-1 fica separada da restante placa e a corrente pode ser medida em série. A medição inclui o *chip*, a memória *flash* e o cristal do módulo, mas não inclui o regulador, o LED de alimentação nem o conversor USB-UART da placa (Figura 1).

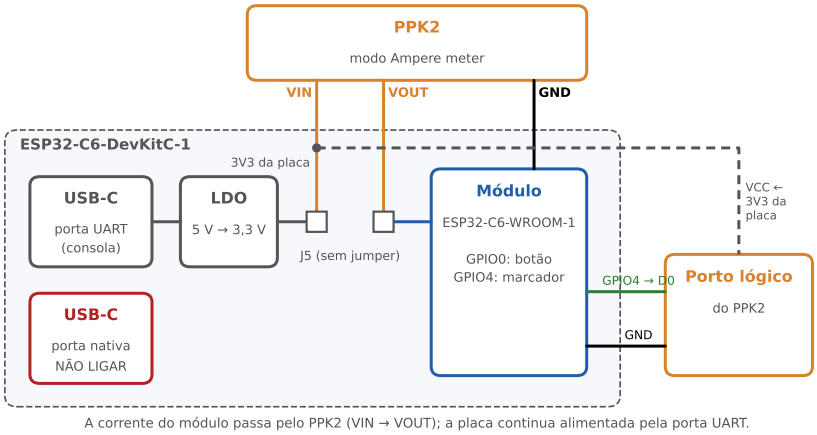


Figura 1: Ligação do PPK2 (modo amperímetro) à ESP32-C6-DevKitC-1

1. Com todos os cabos desligados, retire o *jumper* J5.
2. Ligue o VIN do PPK2 ao pino de J5 do lado do regulador (3V3 da placa), o VOUT ao pino de J5 do lado do módulo e o GND do PPK2 a um GND da placa. A ligação de GND é necessária mesmo em modo amperímetro. Se a placa não arrancar, ou se a corrente medida for próxima de zero, confirme que VIN e VOUT não estão trocados.
3. No porto lógico, ligue D0 ao GPIO4, GND ao GND da placa e VCC ao 3V3 da placa, do lado do regulador, para que a pequena corrente do porto lógico não seja somada à do módulo.
4. Ligue a porta **USB DATA/POWER** do PPK2 ao computador, abra a aplicação *Power Profiler* do *nRF Connect for Desktop* e selecione o PPK2 (**Select Device**). Escolha **Ampere meter** e ative **Enable power output**: em modo amperímetro é este comando que fecha o caminho entre VIN e VOUT.
5. Alimente a placa pela porta USB-UART. Não ligue nenhum cabo à porta USB nativa durante as medições: o seu circuito consome corrente e, como se explica no alerta do Trabalho Prático, a ligação perde-se quando o *chip* dorme.
6. Use 100 kS/s para observar fases curtas e uma taxa menor (por exemplo, 10 kS/s) para gravações longas, e ative o canal digital D0.
7. Selecione cada fase no gráfico: o painel **Selection** indica a corrente média (*average*), o valor máximo (*max*), a duração (*time*) e a carga (*charge*); o painel **Window** indica os mesmos valores para toda a janela visível.

Nas fases de sono, calcule a média sobre janelas de pelo menos 1 s: as mudanças de gama do PPK2 produzem picos breves que não correspondem a consumo real. Para relacionar as medições com a

autonomia usam-se as relações seguintes, em que Q_i é a carga de cada fase e T é o período de um ciclo completo:

$$Q = \bar{I}\, t \qquad \bar{I}_{ciclo} = \frac{\sum_i Q_i}{T} \qquad \text{Autonomia (h)} = \frac{C_{bateria}\ \text{(mAh)}}{\bar{I}_{ciclo}\ \text{(mA)}}$$

5 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na API de Referência dos modos de *sleep* do ESP32-C6, na ficha técnica do ESP32-C6 e no ESP32-C6 Technical Reference Manual, usando o ESP-IDF v6.1.

Os projetos desta aula não incluem o ficheiro `sdkconfig`, que é gerado na primeira compilação: selecione o ESP32-C6 como *target* antes de compilar. Todos incluem o ficheiro `ppk_marker.h`, que coloca o GPIO4 a 1 enquanto o programa corre e a 0 enquanto o *chip* dorme. Ligado à entrada D0 do porto lógico, este **marcador de fase** permite selecionar com exatidão cada fase no gráfico do PPK2. Os gráficos de referência deste guião foram obtidos com o ESP-IDF v5.5.1; os seus valores podem ser diferentes.

> **ALERTA IMPORTANTE!** Nos vários exemplos desta aula, ***opte pela programação do microcontrolador através da UART disponível na ficha USB-C alternativa*** (enumerada tipicamente como `/dev/ttyUSB0`). Quando o ESP32-C6 entra nos modos de *deep-sleep* ou *light-sleep*, o SoC desativa ou desliga os subsistemas dos quais depende a interface USB/JTAG integrada no *chip*, o que, na prática, o desconecta do computador. A porta UART alternativa é servida por um conversor USB-UART externo, que não depende da alimentação do SoC, pelo que a entrada nos modos de *deep-sleep* ou *light-sleep* não a afeta.

5.1 Modo ativo

A Figura 2 mostra, apenas como referência, o consumo da aplicação da Aula 5, que lê o sensor DHT20 e escreve no cartão microSD a cada segundo, sem modos de sono (não é necessário montar o circuito). O consumo médio em "repouso" é de 34,49 mA, com picos de até 40,94 mA, e durante a leitura e a escrita, com uma duração de 79,78 ms, é de 42,16 mA, com picos de até 72,16 mA. Mesmo sem os rádios ativos, este consumo é incompatível com uma autonomia elevada.

5.2 Modo *light-sleep*

Durante o *light-sleep* a CPU, a maioria dos periféricos e o oscilador principal são suspensos. O sistema LP permanece ativo com o seu oscilador lento, que o *timer* usa para contar o tempo e acordar o processador.

5.2.1 Utilização do timer como fonte de *wake-up*

Descarregue o ficheiro `ls_wktimer.zip` e descompacte-o no seu diretório de trabalho.

> **QUESTÃO 1:** Abra o projeto no VSCode e estude o ficheiro `main.c`. Qual a duração do período de *light-sleep* configurado? Compile o programa, descarregue-o para o SoC e abra a consola: deverá observar um *output* semelhante ao seguinte. Altere depois o intervalo entre *wake-ups* e verifique o funcionamento.

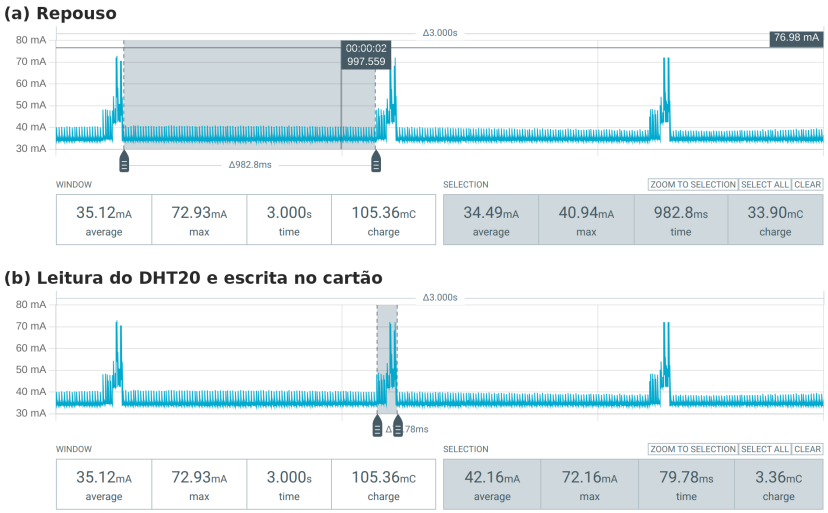


Figura 2: Consumo de referência da aplicação da Aula 5, sem modos de sono

```
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
going for a nap
napped for 999
going for a nap
napped for 999
```

A Figura 3 mostra o consumo de referência do programa original. Durante os 988 ms de *light-sleep* o consumo médio é de 239,19 µA, com picos de até 259,83 µA. Enquanto o programa corre, o consumo médio sobe para 26,25 mA, com picos de até 34,39 mA, durante apenas 3,65 ms, o que corresponde a uma carga de 95,81 µC por ciclo.

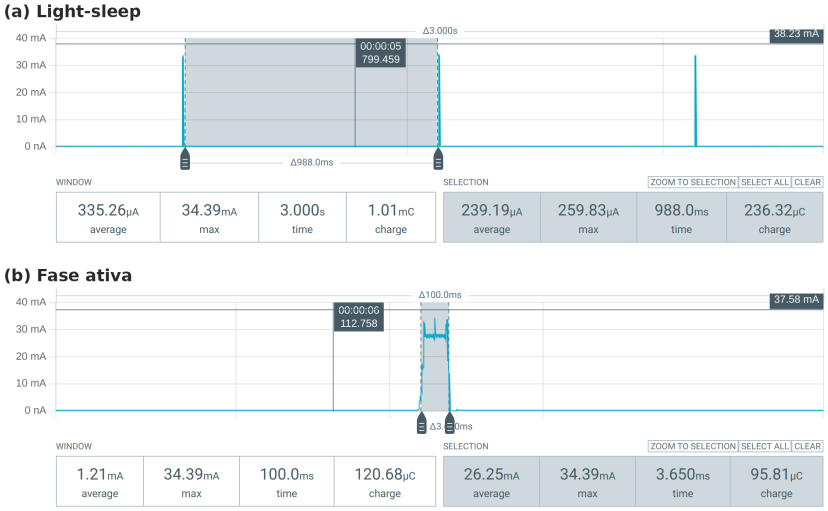


Figura 3: Consumo de referência do projeto `ls_wktimer`

QUESTÃO 2: Com o programa original e o PPK2 ligado como na Figura 1, meça a corrente média no sono, a corrente média e a duração da fase ativa (use o marcador

D0), a carga de cada fase e a corrente média ao longo de 10 s. Registe os valores numa tabela e compare-os com os de referência e com o valor da ficha técnica para o *light-sleep* (Tabela 2). Que componentes do módulo podem explicar a diferença face à ficha técnica?

QUESTÃO 3: Consulte a secção *UART Output Handling* da documentação dos *Sleep Modes* e justifique a invocação da função `console_flush()` antes de `esp_light_sleep_start()` e no fim do ciclo de `app_main()`. No ESP-IDF v6.1, o que acontece por omissão aos caracteres que ainda estão por transmitir quando o *chip* entra em *light-sleep*? E em *deep-sleep*?

QUESTÃO 4: Comente as chamadas a `printf()` e a `console_flush()`, mantendo a medição do tempo, e repita as medições da fase ativa da Questão 2. Em quanto diminuem a duração da fase ativa, a carga por ciclo e a corrente média? Relacione a diferença com o número de caracteres enviados a 115 200 bit/s.

Opcional: reponha as mensagens, retire as chamadas a `console_flush()` e acrescente, antes do ciclo, a linha seguinte. O que observa na consola e no PPK2?

```
esp_sleep_set_console_uart_handling_mode(ESP_SLEEP_ALWAYS_DISCARD_UART);
```

QUESTÃO 5: No programa original, a CPU fica apenas parada durante o *light-sleep*. Para que o ESP-IDF a possa desligar, a aplicação tem de reservar memória para guardar o contexto da CPU, chamando `esp_sleep_cpu_retention_init()` no início de `app_main()`. Os periféricos digitais só podem ser desligados se a CPU também o for.

a) Acrescente essa chamada e repita as medições da Questão 2.

b) Ative depois a opção `CONFIG_PM_POWER_DOWN_PERIPHERAL_IN_LIGHT_SLEEP` no `menuconfig` (pode procurá-la pelo nome) e repita as medições. Compare as correntes em *light-sleep* das três versões entre si e com o valor de 35 µA da ficha técnica. A duração da fase ativa alterou-se? Porquê?

c) Com esta opção, que função de *wake-up* usada no projeto `ls_wkgpio` deixa de estar disponível, e que função a substitui?

Se o marcador D0 deixar de ser fiável com esta opção, identifique as fases apenas pela corrente.

5.2.2 Utilização de GPIO como fonte de *wake-up*

Nesta parte iremos usar um botão de pressão, configurado como na Aula 3 (*pulled-up*) e ligado ao GPIO0. Descarregue o ficheiro `ls_wkgpio.zip` e descompacte-o no seu diretório de trabalho.

QUESTÃO 6: Abra o projeto no VSCode e estude o ficheiro `main.c`. Explique qual a função do seguinte bloco de código:

```
if (gpio_get_level(INPUT_PIN) == 0) {
            printf("please release button\n");
            console_flush();
            do {
                vTaskDelay(pdMS_TO_TICKS(10));
            } while (gpio_get_level(INPUT_PIN) == 0);
}
```

QUESTÃO 7: Compile o programa, descarregue-o para o SoC e abra a consola. Ao premir o botão, o SoC deverá acordar do *light-sleep* e mostrar um *output* semelhante ao seguinte. Meça com o PPK2 a corrente média em *light-sleep* e a duração e a carga da fase ativa que se segue a um *wake-up*. A fase ativa é mais longa do que no projeto anterior. Porquê? A corrente em *light-sleep* é igual à medida na Questão 2?

```
going for a nap
woke up after 695 ms, reason: button
please release button
going for a nap
woke up after 144 ms, reason: button
please release button
```

Nas medições de referência (Figura 4), o consumo em *light-sleep* não vai além de 181,6 µA, ficando-se em média pelos 160,55 µA e 161,14 µA. Os intervalos medidos, de 687,1 ms e 136,4 ms, são muito próximos dos reportados na consola (695 ms e 144 ms). Na fase ativa, o consumo médio sobe para 21,81 mA, com picos de até 34,99 mA, durante 126,1 ms.

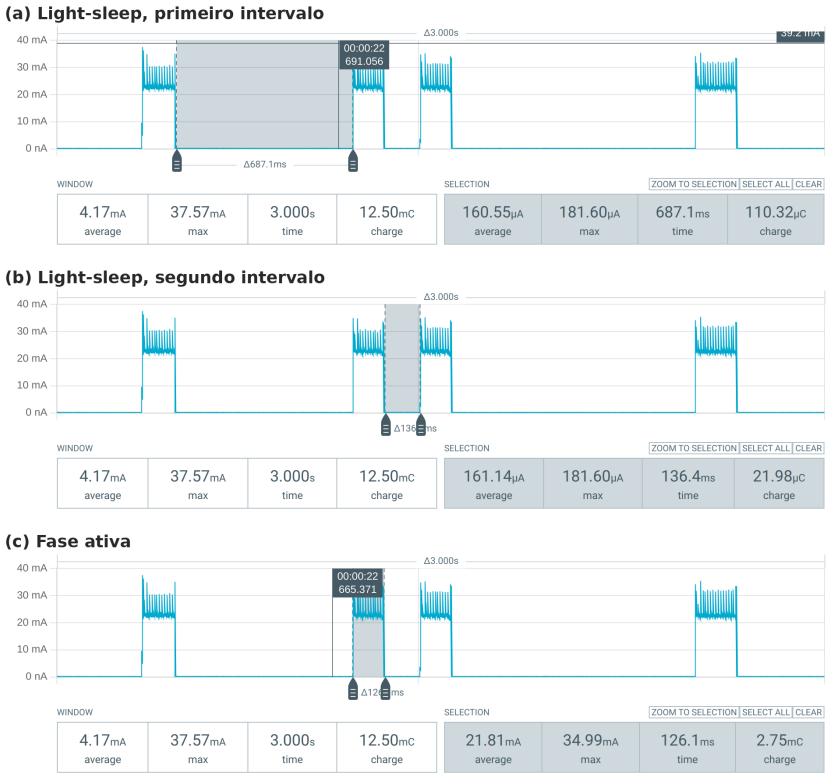


Figura 4: Consumo de referência do projeto `ls_wkgpio`

No ESP32-C6, o ESP-IDF isola por omissão todos os GPIO durante o *light-sleep* (opção `CONFIG_ESP_SLEEP_GPIO_RESET_WORKAROUND`): desliga as entradas, as saídas e as resistências de *pull-up* e *pull-down*. Ficam excluídos os pinos configurados como fonte de *wake-up* com `gpio_wakeup_enable()` e aqueles em que se chama `gpio_sleep_sel_dis()`, que mantêm durante o sono a configuração normal. É por isso que `ppk_marker.h` chama `gpio_sleep_sel_dis()` para o GPIO4.

QUESTÃO 8: Avalie o consumo de uma resistência de *pull-up* durante o sono.

a) No projeto `ls_wktimer`, configure o GPIO0 como entrada com *pull-up* interno, como em `ls_wkgpio`, mas sem o configurar como fonte de *wake-up*. Meça a corrente em *light-sleep* com o botão solto e com o botão premido.

b) Acrescente `gpio_sleep_sel_dis(GPIO_NUM_0)` depois de `gpio_config()` e repita as medições. Estime o valor da resistência de *pull-up* interna a partir da diferença de corrente e compare-o com o valor indicado na ficha técnica.

c) Que corrente consumiria uma resistência de *pull-up* externa de 10 kohm com o botão premido? Compare-a com a corrente do módulo em *light-sleep*.

5.3 Modo *deep-sleep*

O *deep-sleep* desliga o processador, a RAM principal e a maioria dos periféricos, mantendo apenas o sistema LP. O consumo desce para a ordem dos microamperes, mas o regresso à operação ativa implica um reinício completo do sistema.

5.3.1 Utilização do timer como fonte de *wake-up*

Descarregue o ficheiro `ds_wktimer.zip` e descompacte-o no seu diretório de trabalho.

QUESTÃO 9: Abra o projeto no VSCode e estude o ficheiro `main.c`. Qual o funcionamento esperado para este programa? Compile-o, descarregue-o e abra a consola. Deveria observar um *output* semelhante ao seguinte, em que a variável de contagem é incrementada a cada ciclo. Contudo, isso não ocorre. Porquê? **Pista:** consulte a documentação dos *Sleep Modes*.

```
I (251) main_task: Calling app_main()
going to sleep. woken up 0
ESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x5 (SLEEP_WAKEUP),boot:0x6f (SPI_FAST_FLASH_BOOT)
...
I (24) boot: ESP-IDF v6.1 2nd stage bootloader
...
I (251) main_task: Calling app_main()
going to sleep. woken up 1
```

QUESTÃO 10: Altere o programa de modo a que a variável de contagem seja preservada entre *wake-ups* sucessivos e teste o seu programa.

A Figura 5 mostra o consumo de referência deste programa. Em *deep-sleep*, o consumo cai para 8,26 µA em média, atingindo no máximo 10 µA. Em cada *wake-up*, o consumo é em média de 30,62 mA, com um máximo de 50,04 mA, durante 256,8 ms, o que corresponde a uma carga de 7,87 mC. Com um período de cerca de 3 s, a corrente média do ciclo é de 2,42 mA, muito superior aos cerca de 335 µA do programa de *light-sleep* com período de 1 s: para períodos curtos, a carga gasta em cada arranque domina o consumo.

QUESTÃO 11: Meça a corrente média em *deep-sleep* e a duração e a carga de cada *wake-up*. O marcador D0 só passa a 1 quando `app_main()` começa: que fração da fase ativa decorre antes disso, e em que etapas do arranque é gasta?

QUESTÃO 12: Com o *deep-sleep*, cada *wake-up* custa mais carga do que com o *light-sleep*, mas a corrente durante o sono é menor. Sejam Q_{ls} e Q_{ds} as cargas de um

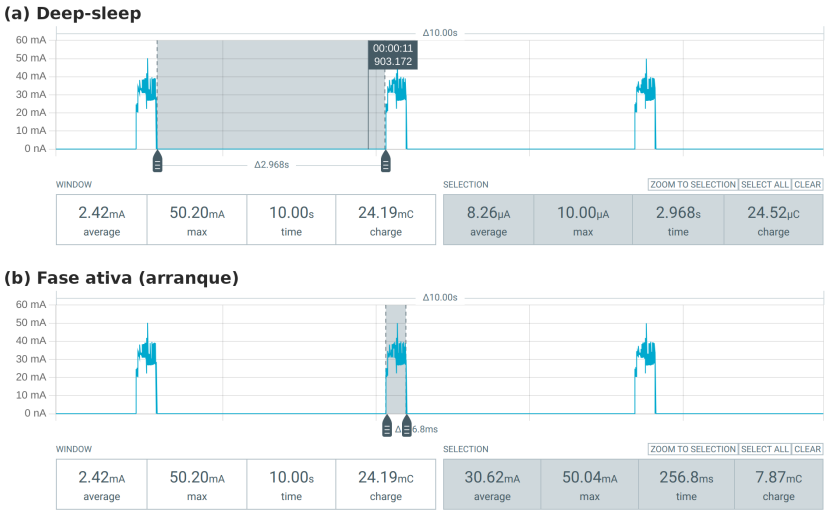


Figura 5: Consumo de referência do projeto `ds_wktimer`

wake-up e I_{ls} e I_{ds} as correntes durante o sono em *light-sleep* e em *deep-sleep*. Mostre que o *deep-sleep* só compensa para períodos superiores a

$$T^* = \frac{Q_{ds} - Q_{ls}}{I_{ls} - I_{ds}}$$

e calcule T^* com os valores de referência e com os seus. O projeto prevê períodos de amostragem entre 1 s e 3600 s: em que parte desse intervalo deve ser usado cada modo?

QUESTÃO 13: Reduza a duração do arranque após o *deep-sleep* ativando, no `menuconfig`, as opções `CONFIG_BOOTLOADER_SKIP_VALIDATE_IN_DEEP_SLEEP`, `CONFIG_BOOTLOADER_LOG_LEVEL_NONE` e `CONFIG_LOG_DEFAULT_LEVEL_WARN`. Como o *bootloader* também muda, volte a descarregar o projeto completo. Meça de novo a duração e a carga de cada *wake-up* e recalcule T^*. Que garantia se perde com a primeira opção?

5.3.2 Utilização do sinal EXT1 como fonte de *wake-up*

Com o EXT1, o sistema LP monitoriza o nível dos LP GPIO selecionados durante o sono e acorda o *chip* quando a condição é satisfeita. A função `esp_sleep_enable_ext1_wakeup_io()` define se o *chip* acorda quando qualquer dos pinos estiver a 1 (`ESP_EXT1_WAKEUP_ANY_HIGH`) ou a 0 (`ESP_EXT1_WAKEUP_ANY_LOW`); no ESP32-C6, chamadas sucessivas com pinos diferentes permitem escolher um nível diferente para cada pino. Depois de acordar, a função `esp_sleep_get_ext1_wakeup_status()` indica que pinos provocaram o *wake-up*.

Nesta parte iremos usar o botão ligado ao GPIO0. Descarregue o ficheiro `ds_wkext1.zip` e descompacte-o no seu diretório de trabalho.

QUESTÃO 14: Abra o projeto no VSCode e estude o ficheiro `main.c`. Compile o programa e teste-o premindo o botão. Cada pressão deverá provocar um arranque completo, seguido de uma mensagem com o contador incrementado:

```
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 0
```

```
ESP-ROM:esp32c6-20220919
...
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 1
```

Nas medições de referência (Figura 6), o modo *deep-sleep* apresenta um consumo médio de 9,64 µA, com um valor máximo de 11,98 µA, e cada *wake-up* consome em média 30,79 mA, com um máximo de 50,36 mA, durante 271,2 ms.

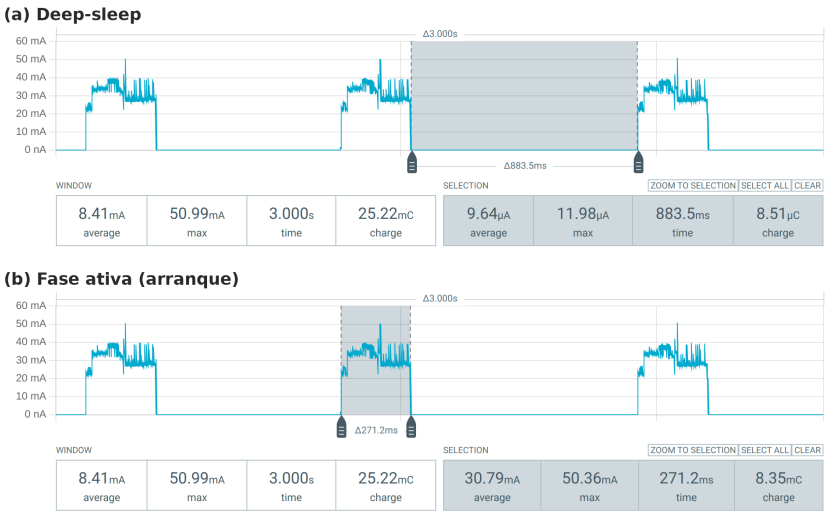


Figura 6: Consumo de referência do projeto `ds_wkext1`

QUESTÃO 15: Analise o consumo do *wake-up* por EXT1.

a) Meça a corrente média em *deep-sleep* e compare-a com a do projeto `ds_wktimer`. A que se deve a diferença?

b) Comente a chamada a `esp_sleep_pd_config(ESP_PD_DOMAIN_RTC_PERIPH, ESP_PD_OPTION_ON)` e repita a medição. O botão continua a acordar o *chip*? Consulte a documentação de `esp_sleep_enable_ext1_wakeup_io()` em `esp_sleep.h` e explique o resultado.

c) Mantenha o botão premido durante alguns segundos. O que observa na consola e no PPK2? Porquê?

QUESTÃO 16: Monte um segundo botão de pressão, ligado ao GPIO5 (um LP GPIO), e configure-o com *pull-up* interno, como o primeiro. Altere o programa de modo a que só conte um *wake-up* quando ambos os botões estiverem premidos em simultâneo. Como o ESP32-C6 não suporta o modo `ESP_EXT1_WAKEUP_ALL_LOW`, acorde com `ESP_EXT1_WAKEUP_ANY_LOW`, verifique o estado dos dois pinos com `rtc_gpio_get_level()` e, se só um estiver premido, volte a adormecer, contando estes *wake-ups* falsos numa segunda variável. Atenção ao comportamento observado na Questão 15 c). Meça a carga de um *wake-up* falso e comente o custo desta solução.

5.4 Estimativa de autonomia

A Tabela 3 reúne os valores de referência apresentados neste guião.

Tabela 3: Valores de referência medidos com o PPK2 (ESP-IDF v5.5.1)

Fase	Corrente média	Duração	Carga
Leitura do DHT20 e escrita no cartão (Aula 5)	42,16 mA	79,78 ms	3,36 mC
Light-sleep (*wake-up* por *timer*)	239,19 µA	988 ms	236,32 µC
Fase ativa após *light-sleep*	26,25 mA	3,65 ms	95,81 µC
Deep-sleep (*wake-up* por *timer*)	8,26 µA	2,968 s	24,52 µC
Fase ativa após *deep-sleep* (*timer*)	30,62 mA	256,8 ms	7,87 mC
Deep-sleep (*wake-up* por EXT1)	9,64 µA	883,5 ms	8,51 µC
Fase ativa após *deep-sleep* (EXT1)	30,79 mA	271,2 ms	8,35 mC

QUESTÃO 17: Considere um nó do projeto que lê o DHT20 e escreve a amostra no cartão com um período T, gastando em cada leitura a carga indicada na Tabela 3, e que dorme entre leituras.

a) Usando as suas medições (ou, na falta delas, os valores de referência), calcule a corrente média para $T = 10$ s e para $T = 600$ s, adormecendo em *light-sleep* e em *deep-sleep*.

b) Calcule a autonomia correspondente com uma bateria de 2000 mAh, admitindo um regulador linear, em que a corrente pedida à bateria é igual à corrente do módulo.

c) Indique três razões pelas quais estas estimativas são otimistas. Considere o que a medição em J5 não inclui, o consumo dos sensores e do cartão durante o sono e as comunicações sem fios.

d) O nó tem de manter uma interface BLE disponível. Que consequência tem esta exigência na escolha entre *light-sleep* e *deep-sleep*?

6 Acknowledgements

Originally authored by Paulo C. Bartolomeu